Crowned Kind-Polymorphic Forest Reflection

A proposed Contender 11 for BusyLean

OpenAI research package prepared for the BusyLean repository

October 2026

Abstract

At the pinned revision `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`, the current BusyLean entry is Contender 10, obtained by reflecting a kind-polymorphic extension of System F_ω over Contender 9's numerical family. This article proposes a structurally stronger, still constructive entry. First, the unary oracle of Contender 10 is lifted back to a countable family by the repository's certified 2-adic pairing. Second, the resulting kind-polymorphic reflector is installed at every node of a well-founded W-tree. Third, the forest is run along the repository's self-generated tree index, which joins every $W\mathbb{N}$ tree denoted by the earlier W-language. Finally, an input-diagonal "crown" of $n+1$ additional reflection nodes is placed above that index.

The proposed family is

$$\mathsf{family}_{10}(n) = F_{10}(F_{10}(n)), \qquad F_{11}(n) = \mathcal{S}^{(11)}_{\mathsf{family}_{10}}\big(\mathsf{Crown}_{n+1}(I), n\big),$$

and the proposed closed number is

$$C_{11} = F_{11}(F_{11}(42)).$$

The comparison is exact and needs no ordinal calibration: $\mathsf{family}_{10}(n) < F_{11}(n)$ for every n, hence $C_{10} = \mathsf{family}_{10}(42) < C_{11}$. Stronger structural statements show that every crown edge is strict, every accepted depth-bounded kind-polymorphic program at the root is exceeded, and every tree denoted by the old W-language is eventually dominated.

The package supplies Lean modules, an axiom/dependency audit, integration tooling, a finite independent reference model, and reproducibility evidence. The packaging environment did not contain Lean 4.34.0 or `lake`; therefore the formal files are a concrete kernel-checkable proposal, not a claim of a completed kernel build. Static auditing and 670 finite structural checks pass; the remaining certification and timing obligations are stated explicitly.

Certification status. The mathematical comparison and the proposed Lean proof are given in full. The PDF, source audit, package hashes, and finite reference tests were produced in the present environment. Lean elaboration, `#print axioms`, transitive closure checking, and the 15/60 second budgets remain to be run in a pinned Lean 4.34.0 checkout. Nothing in this article should be read as silently substituting Python tests for the Lean kernel.

Contents

1 Research target **3**

1.1 The actual baseline 3

1.2 Contest obligations 4
1.3 What "significantly exceeds" means here 4

2 The inherited semantic interfaces 5
2.1 Kind-polymorphic reflection 5
2.2 Packing a family into one unary oracle 5
2.3 The self-generated W-tree index 5

3 The new family-valued reflector 6

4 Installing the reflector on a well-founded forest 7

5 The diagonal crown 7
5.1 Definition 7
5.2 Why the height is input-diagonal 8

6 The proposed family and closed number 8

7 Exact comparison with Contender 10 9
7.1 The index already beats the seed 9
7.2 Adding the crown 9
7.3 Closing the diagonal 9

8 Stronger separation theorems 10
8.1 Root-program reflection 10
8.2 Eventual domination of every old-language tree 10
8.3 A strict semantic ladder 10

9 Constructivity and totality 10

10 Why this is a qualitative advance 11
10.1 From one reflection to a reflection forest 11
10.2 What is proved, and what is heuristic 11
10.3 No claim of maximality 12

11 No-laziness and dependency hygiene 12

12 Formalization inventory 12

13 Validation performed for this package **13**
13.1 Static Lean-source audit . . . 13
13.2 Independent finite reference model . . . 13
13.3 Outstanding kernel checks . . . 14

14 Integration and likely elaboration points **14**

15 Potential objections and responses **15**
15.1 “Is the crown merely padding?” . . . 15
15.2 “Could Contender 10 write the same computation as one program?” . . . 15
15.3 “Does the old W-language index limit the advance?” . . . 15
15.4 “Where is the exact proof-theoretic ordinal?” . . . 15

16 Further research program **15**
16.1 Immediate certification . . . 15
16.2 A kind-polymorphic W-language . . . 15
16.3 Typed family oracles . . . 16
16.4 Self-generated kind indices . . . 16
16.5 Certified majorants and monotone reflection . . . 16
16.6 Exact calibration . . . 16
16.7 Enumeration compression . . . 16

17 Conclusion **17**

A Complete proposed definition module **17**

B Complete proposed comparison module **18**

C Reproduction checklist **21**

D Artifact map **21**

1 Research target

1.1 The actual baseline

The target is not the original Coq Contender 5 and not one of the earlier unmerged proposals. The inspected BusyLean revision is `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`. At that revision, the repository defines

```
def family9 (n : Nat) : Nat := F9 (F9 n)
def F10 (n : Nat) : Nat := Fk.reflectFk family9 n
def contender_10 : Nat := F10 (F10 42)
```

and proves `contender_9_lt_contender_10`. The reflected object language is a kind-polymorphic extension of System F_ω with a total normalizer justified by reducibility candidates. A bounded syntax list and a sound checker make its finite numerical maximum a computable natural number.

Contender 10 is already a substantial entry. A serious successor must not merely add one, multiply by a fixed constant, or repeat the named scalar. The new construction below changes the *shape of reflection*: the stronger object language is reflected at every node of a well-founded forest, and the forest is traversed through a self-generated index plus a diagonal crown.

1.2 Contest obligations

The repository rules impose five constraints that matter directly here.

1. The result must be a closed computable `Nat`. Uncomputable Busy Beaver functions, `noncomputable`, `partial`, `unsafe`, external implementations, and opaque placeholders are excluded.
2. The comparison with the preceding entry must be a formal strict inequality.
3. The permitted axiom envelope is the one already used by BusyLean: `propext` and `Quot.sound`; classical choice and custom axioms are excluded.
4. The new definition and proof phases have separate 15-second and 60-second budgets once the accepted predecessor is precompiled.
5. The named scalar `contender_10` may not be used as a shortcut. Reusing the preceding *construction* as an oracle family is permitted when it is embedded in a qualitatively stronger construction.

The proposed source is organized around these constraints. The definition module imports only the previous definition module. It defines the old construction uniformly as `family10 n := F10 (F10 n)`, but its executable syntax contains no occurrence of `contender_10`. The proof and audit are separate modules.

1.3 What "significantly exceeds" means here

A single scalar inequality is necessary but not, by itself, persuasive. We therefore use four exact witnesses of structural separation.

Witness	Exact content
Uniform family bound	$\mathsf{family}_{10}(n) < F_{11}(n)$ for every n.
Crown ladder	Every consecutive crown level is strictly larger at every input. At input 42 there are 43 such edges; at the outer input there are $F_{11}(42) + 1$ edges.
Program reflection	Every accepted $\mathrm{F}^{\mathsf{k}}_{\omega}$ program of depth at most n, relative to the preceding crown level, is below $F_{11}(n)$.
Tree cofinality	Every $W\mathbb{N}$ tree denoted by the earlier W-language is strictly below the new family from an explicit threshold onward.

None of these statements relies on a conjectural ordinal label.

2 The inherited semantic interfaces

2.1 Kind-polymorphic reflection

Write $V_f(n,t)$ for `Fk.fvalue f n t`: type-check t with fuel n; if its type is the Church numeral type, erase and normalize it relative to the unary oracle f, then decode the normal form; otherwise return zero. Let T_n be the finite list `Fk.ctmsUpTo n`. Contender 10 uses

$$R_{10}(f,n) = n + 1 + \max\left(f(n), \max_{t \in T_n} V_f(n,t)\right).$$

The repository proves three facts that are crucial for reuse:

1. the checker is sound;
2. checked terms erase to strongly normalizing terms;
3. $R_{10}(f,n)$ exceeds both $f(n)$ and every listed program value.

The proposed successor does not alter the object language, checker, normalizer, or decoder. It changes the oracle interface from one function to a countable family and then iterates that reflection through a W-tree.

2.2 Packing a family into one unary oracle

For $k > 0$, write uniquely

$$k = 2^i(2m+1).$$

The repository's functions `v2`, `unpair2`, and `packFamily` implement

$$\mathrm{code}(i,m) = 2^i(2m+1), \qquad \mathrm{pack}(f)(k) = f(\mathrm{unpair}_2(k)).$$

It proves the exact decoder theorem

$$\mathrm{unpair}_2(2^i(2m+1)) = (i,m),$$

so

$$\mathrm{pack}(f)(2^i(2m+1)) = f(i,m).$$

This matters because a kind-polymorphic program at a forest node can call any child i at any numerical input m using an ordinary unary oracle call. No uncomputable family oracle is introduced.

2.3 The self-generated W-tree index

The earlier W-language gives a total interpretation of raw terms into values that include $W\mathbb{N}$ trees. BusyLean defines

$$\begin{aligned} \mathrm{defTree}(t) &:= \text{the } W\mathbb{N} \text{ tree denoted by } t, \\ \mathrm{treesOfDepth}(k) &:= \mathrm{node}\big(i \mapsto \mathrm{defTree}(\mathrm{nthD}(\mathrm{allTerms}(k), i))\big), \\ I &:= \mathrm{node}(\mathrm{treesOfDepth}). \end{aligned}$$

Thus child k of I joins every tree in the depth-k enumeration. The first enumeration is empty, so its default term denotes a leaf. This gives an exact three-step route

$$\text{seed value} < \mathcal{S}^{(11)}(\mathsf{leaf}) < \mathcal{S}^{(11)}(\text{treesOfDepth}(0)) < \mathcal{S}^{(11)}(I),$$

which will be the core of the comparison.

3 The new family-valued reflector

Definition 3.1 (Family reflection)**.** For a family $f : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, define

$$\mathcal{R}_{11}(f, n) = n + 1 + \max\left(\max_{i \le n} f(i, n), \max_{t \in T_n} V_{\text{pack}(f)}(n, t)\right).$$

In Lean this is `reflect11`.

```
def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (Fk.fvalue (packFamily fs) n)
      (Fk.ctmsUpTo n))
```

Every operation in this expression is total. The two maxima are over finite lists or finite prefixes. The only potentially subtle component is `Fk.fvalue`, whose totality is inherited from the formal strong normalization theorem.

Lemma 3.2 (Visible child bound)**.** *If $i \le n$, then $f(i, n) < \mathcal{R}_{11}(f, n)$.*

Proof. The prefix maximum is at least $f(i, n)$. It is at most the outer maximum, and $\mathcal{R}_{11}$ adds $n + 1$, which is positive. □

Lemma 3.3 (Program bound)**.** *If $t \in T_n$, then*

$$V_{\text{pack}(f)}(n, t) < \mathcal{R}_{11}(f, n).$$

Proof. The listed program value is at most the finite program maximum, hence at most the outer maximum. Again the positive summand makes the inequality strict. □

Lemma 3.4 (Input bound)**.** *For every f, n, $n < \mathcal{R}_{11}(f, n)$.*

Proof. $\mathcal{R}_{11}(f, n) = n + 1 + M$ for a natural number M. □

These lemmas are deliberately order-free in the oracle. Reflection is not monotone in an arbitrary oracle: a program may inspect an oracle result and branch in an anti-monotone way. All comparisons in this package therefore use *explicit inclusion*—a child occurs in the finite prefix, or a program occurs in the finite syntax list—rather than a false monotonicity principle.

4 Installing the reflector on a well-founded forest

Definition 4.1 (Forest stage)**.** For a seed family s, define by structural recursion on a $W\mathbb{N}$ tree

$$\mathcal{S}_s^{(11)}(\mathsf{leaf}, n) = \mathcal{R}_{11}(s, n),$$
$$\mathcal{S}_s^{(11)}(\mathsf{node}(g), n) = \mathcal{R}_{11}(i \mapsto \mathcal{S}_s^{(11)}(g(i), \cdot), n).$$

```
def stage11 (seed : W.Family) : BigNumber.WTree Nat → Nat → Nat
  | .leaf => reflect11 seed
  | .node g => reflect11 (fun i => stage11 seed (g i))
```

This is the same well-founded recursion pattern already accepted for the repository's Contenders 6 and 9. At a node, the unary object-language oracle is the packed family of all child stages. A program can choose a child and an input dynamically.

Lemma 4.2 (Strict parent edge)**.** *If $i \le n$, then*

$$\mathcal{S}_s^{(11)}(g(i), n) < \mathcal{S}_s^{(11)}(\mathsf{node}(g), n).$$

Proof. At the parent, the oracle family is $j \mapsto \mathcal{S}_s^{(11)}(g(j), \cdot)$. Apply the visible child bound at index i. □

Corollary 4.3 (Every stage exceeds its input)**.** *For every seed s, tree a, and input n,*

$$n < \mathcal{S}_s^{(11)}(a, n).$$

Proof. Case split on a and apply the input bound for $\mathcal{R}_{11}$. □

The parent-edge theorem is the engine behind both the self-generated index and the crown. It is unusually robust: no semantic comparison between two oracles, and no analysis of program behavior, is required.

5 The diagonal crown

5.1 Definition

For a tree a, put

$$\mathsf{Crown}_0(a) = a, \qquad \mathsf{Crown}_{k+1}(a) = \mathsf{node}(i \mapsto \mathsf{Crown}_k(a)).$$

Each crown node repeats one child at every index. In particular, index zero is visible at every input.

```
def crown11 : Nat → BigNumber.WTree Nat → BigNumber.WTree Nat
  | 0, a => a
  | k + 1, a => .node fun _ => crown11 k a
```

Lemma 5.1 (Crown step)**.** *For all s, a, k, n,*

$$\mathcal{S}_s^{(11)}(\mathsf{Crown}_k(a), n) < \mathcal{S}_s^{(11)}(\mathsf{Crown}_{k+1}(a), n).$$

Proof. The left-hand tree is child zero of the right-hand node. Since $0 \le n$, the strict parent-edge theorem applies. □

Corollary 5.2 (Positive crown)**.** *For all k, n,*

$$\mathcal{S}_s^{(11)}(a, n) < \mathcal{S}_s^{(11)}(\mathsf{Crown}_{k+1}(a), n).$$

Proof. Chain $k + 1$ crown-step inequalities. The supplied Lean proof performs this chaining by induction on k. □

5.2 Why the height is input-diagonal

A fixed crown of height 43 would strengthen the scalar at input 42 but would not define a uniformly expanding family. We instead use height $n + 1$. This has three benefits.

1. The uniform theorem records a growing number of explicit semantic reflection levels, not a hard-coded scalar embellishment.
2. At the advertised inner input 42, there are exactly 43 strict crown edges above the self-generated index.
3. At the outer input $F_{11}(42)$, the closed contender traverses $F_{11}(42) + 1$ fresh crown edges. This is a diagonal amplification whose height is itself already the new family value.

The crown is not the only source of growth: beneath it lies the entire self-generated forest. Its purpose is to provide a clean, exact, inspectable margin in addition to the more complex forest semantics.

6 The proposed family and closed number

Define the predecessor uniformly by

$$\mathsf{family}_{10}(n) = F_{10}(F_{10}(n)).$$

By definitional equality, $\mathsf{family}_{10}(42) = C_{10}$. Let the seed family ignore its child index and return family_{10}:

$$s(i, n) = \mathsf{family}_{10}(n).$$

Then define

$$F_{11}(n) = \mathcal{S}_s^{(11)}(\mathsf{Crown}_{n+1}(I), n), \qquad C_{11} = F_{11}(F_{11}(42)).$$

```
def family10 (n : Nat) : Nat := F10 (F10 n)

def F11 (n : Nat) : Nat :=
  stage11 (fun _ => family10) (crown11 (n + 1) index) n

def contender_11 : Nat := F11 (F11 42)
```

The definition module never names `contender_10`. It reuses `F10`, just as earlier accepted BusyLean entries reuse the previous uniform construction as an oracle. The new scalar is therefore not "the old scalar plus a margin"; it is a new reflection hierarchy whose seed happens to contain the predecessor family.

7 Exact comparison with Contender 10

7.1 The index already beats the seed

Let $s(i, n) = \mathsf{family}_{10}(n)$. At every input n:

1. The leaf stage is $\mathcal{R}_{11}(s, n)$, and child zero of the seed prefix is $s(0, n) = \mathsf{family}_{10}(n)$. Hence
$$\mathsf{family}_{10}(n) < \mathcal{S}_s^{(11)}(\mathsf{leaf}, n).$$
2. The first child of `treesOfDepth 0` is the denotation of the default zero term, which the repository proves is a leaf. Therefore
$$\mathcal{S}_s^{(11)}(\mathsf{leaf}, n) < \mathcal{S}_s^{(11)}(\text{treesOfDepth}(0), n).$$
3. `treesOfDepth 0` is child zero of I, so
$$\mathcal{S}_s^{(11)}(\text{treesOfDepth}(0), n) < \mathcal{S}_s^{(11)}(I, n).$$

Combining these inequalities gives the formal theorem
$$\mathsf{family}_{10}(n) < \mathcal{S}_s^{(11)}(I, n).$$
This proof is intentionally concrete. It does not need to know the height or ordinal represented by I.

7.2 Adding the crown

By the positive-crown theorem with $k = n$,
$$\mathcal{S}_s^{(11)}(I, n) < \mathcal{S}_s^{(11)}(\mathsf{Crown}_{n+1}(I), n) = F_{11}(n).$$
Hence:

Theorem 7.1 (Uniform predecessor bound)**.** *For every $n \in \mathbb{N}$,*
$$\mathsf{family}_{10}(n) < F_{11}(n).$$
The supplied Lean name is ***`family10_lt_F11`****.*

7.3 Closing the diagonal

At $n = 42$, definitional equality gives
$$C_{10} = \mathsf{family}_{10}(42) < F_{11}(42).$$
Every forest stage exceeds its input, so at the outer stage
$$F_{11}(42) < \mathcal{S}_s^{(11)}(\mathsf{Crown}_{F_{11}(42)+1}(I), F_{11}(42)) = F_{11}(F_{11}(42)) = C_{11}.$$
Therefore:

Theorem 7.2 (Required scalar comparison)**.**
$$C_{10} < C_{11}.$$
The supplied Lean name is ***`contender_10_lt_contender_11`****.*

The proof does not unfold a single enormous numeral. It composes the uniform family inequality with the general input bound.

8 Stronger separation theorems

8.1 Root-program reflection

Because

$$\mathsf{Crown}_{n+1}(I) = \mathsf{node}(i \mapsto \mathsf{Crown}_n(I)),$$

the root oracle of $F_{11}(n)$ is the constant family

$$i \mapsto \mathcal{S}_s^{(11)}(\mathsf{Crown}_n(I), \cdot).$$

Consequently every accepted program $t \in T_n$ satisfies

$$V_{\mathrm{pack}(i \mapsto \mathcal{S}_s^{(11)}(\mathsf{Crown}_n(I), \cdot))}(n, t) < F_{11}(n).$$

This is exactly `root_program_lt_F11`. It says more than "the previous value occurs in a maximum": the entire finite fragment of the strong object language, relative to the entire preceding crown level, is reflected at the root.

8.2 Eventual domination of every old-language tree

Suppose a raw W-language term t belongs to the depth-k enumeration, and let i be an index where it occurs. For every $n \geq k + i$, both $k \leq n$ and $i \leq n$. The denoted tree is then a visible child of `treesOfDepth k`; that join is a visible child of I; and I is strictly below the crown. Thus

$$\mathcal{S}_s^{(11)}(\mathrm{defTree}(t), n) < F_{11}(n).$$

The threshold $k + i$ is explicit. The Lean theorem is `defined_tree_eventually_lt11`.

This cofinality result is relative to the earlier W-language's tree output. It does not claim that kind-polymorphic programs themselves generate every tree; the current $\mathrm{F}_\omega^{\mathsf{k}}$ object language returns Church numerals, not W-objects. That distinction is a natural target for a future, stronger contender.

8.3 A strict semantic ladder

For fixed n, set

$$A_k(n) = \mathcal{S}_s^{(11)}(\mathsf{Crown}_k(I), n).$$

Then

$$A_0(n) < A_1(n) < \cdots < A_{n+1}(n) = F_{11}(n).$$

At $n = 42$, the package therefore exhibits a 43-edge strict chain above the already self-generated index. At the final outer input, it exhibits a chain of length $F_{11}(42) + 1$. This is a structural count of reflection nodes, not a claim that successive numerical differences are one; each edge encloses a new finite maximum over object-language computations relative to the lower level.

9 Constructivity and totality

The proposed number is computable in principle for the same reason as the accepted predecessors, plus two elementary structural recursions.

1. **Finite enumeration.** `Fk.ctmsUpTo n` is a finite list constructed recursively from finite lists of kinds, types, and terms.
2. **Decidable checking.** `Fk.infer` is fuel bounded. Rejected terms contribute zero.
3. **Total normalization.** An accepted term at the Church numeral type has a proof of strong normalization, so `Fk.nf` is a total Lean function on the proof-carrying term.
4. **Finite maxima.** Both the syntax maximum and the child-prefix maximum range over finite data.
5. **Well-founded forest recursion.** `stage11` recurses structurally on `WTree`, exactly as the existing `stage9` does.
6. **Primitive crown recursion.** `crown11` recurses on a natural number and constructs a finite unary spine.
7. **Closed specialization.** `contender_11` substitutes the closed numeral 42 into closed total functions.

No semantic oracle is assumed. The only oracle seen by an object-language program is a Lean-computable function generated from recursive calls on strict subtrees. The result is fantastically infeasible, but infeasibility is not noncomputability.

Never evaluate the actual contender. Computability is certified intensionally by total definitions and termination proofs. Attempting to normalize `F11 42` would be both unnecessary and catastrophic. The comparison proofs quantify over symbolic inputs and use finite-maximum inequalities.

10 Why this is a qualitative advance

10.1 From one reflection to a reflection forest

Contender 10 applies the kind-polymorphic reflector once to `family9` at each input. The new construction makes that reflector the local transition at *every node* of a well-founded tree. At a node, an object-language program has uniform access to all child stages through the packed oracle. The node then exceeds all visible children and all accepted programs relative to them.

This is not equivalent to a fixed iteration count written in arithmetic. The shape of the iteration is a W-tree whose branching can be selected by computed indices. The self-generated index ranges over every tree output by the earlier W-language, and the crown adds a separate diagonal sequence of new layers.

10.2 What is proved, and what is heuristic

The following claims are exact consequences of the supplied definitions and proof scripts:

- $\mathsf{family}_{10}(n) < F_{11}(n)$ for every n;
- $C_{10} < C_{11}$;
- each crown edge is strict;

- all root programs in the bounded F^{k}_{ω} enumeration are reflected;
- every W-language-defined tree is eventually dominated.

It is reasonable to describe the move from a single F^{k}_{ω} reflection to F^{k}_{ω} reflection along a self-generated forest as a multiplicative or iteration-level increase in proof-theoretic terms. However, this article does *not* assign an exact ordinal to the kind-polymorphic language, to the forest closure, or to the crowned diagonal. The repository's formal inequalities are stronger evidence than an unsupported ordinal name.

10.3 No claim of maximality

No computable description can be the last word in an unrestricted contest: a new description can reflect or diagonalize over it. The research objective is therefore a transparent and formally comparable advance over the actual incumbent, not a metaphysical "largest computable number."

11 No-laziness and dependency hygiene

The computational dependency graph is designed to satisfy the repository's no-laziness rule.

1. `Contender11.lean` imports `BusyLean.Contender10`, not `BusyLean.Comparison10`.
2. Its executable syntax contains no identifier `contender_10`.
3. The predecessor enters only through `family10 n := F10 (F10 n)`, a uniform family subsequently used as the leaf seed of a new forest operator.
4. The new scalar applies `F11` to its own inner value, rather than adding a large but semantically inert arithmetic constant.

The package includes a kernel-side transitive closure command modeled on the repository's audit. It is intended to establish that `BusyLean.contender_11` avoids every earlier named scalar contender, while still exposing all legitimate dependencies on the generic constructions. A separate static audit strips nested Lean comments and strings before checking that `contender_10` is absent from executable definition syntax.

12 Formalization inventory

Lean declaration	Role
`reflect11`	Family-valued F^{k}_{ω} reflection step.
`stage11`	Well-founded forest recursion installing the reflector at all nodes.
`crown11`	Finite unary spine above a base tree.
`family10`	Uniform predecessor construction; definitionally equals Contender 10 at input 42.
`F11`	Crowned forest family.

Lean declaration	Role
`contender_11`	Closed proposed number.
`oracle_lt_reflect11`	Visible child bound.
`fvalue_lt_reflect11`	Bounded-program bound.
`input_lt_stage11`	Every stage exceeds its input.
`stage11_crown_step_lt`	Every crown edge is strict.
`family10_lt_stage11_index`	Explicit leaf–join–index comparison.
`family10_lt_F11`	Uniform predecessor-family comparison.
`defined_tree_eventually_lt11`	Cofinality over old-language trees.
`root_program_lt_F11`	Root reflects programs relative to the preceding crown.
`contender_10_lt_contender_11`	Required scalar comparison.

The proof module is intentionally short relative to the inherited normalization development. Its substantive work is orchestration: existing soundness and normalization justify `Fk.fvalue`; existing pairing exposes child families; new finite-maximum and tree-edge arguments establish the hierarchy.

13 Validation performed for this package

13.1 Static Lean-source audit

The static audit passed all advertised checks. It verifies, after removing nested comments and strings:

- absence of `noncomputable`, `partial`, `unsafe`, `opaque`, `extern`, `native_decide`, `sorry`, custom `axiom`, and related escape hatches;
- absence of the named scalar `contender_10` from executable `Contender11.lean` syntax;
- separation of definition and proof imports;
- presence of all stated definitions and theorems;
- balanced ASCII delimiters in all new Lean files;
- an exact strict target for the required comparison.

This is useful evidence against packaging mistakes, but it is not a Lean parser or kernel.

13.2 Independent finite reference model

A small Python model reimplements only the structural algebra. It does not attempt System F_ω normalization. The test suite passed 670 assertions:

- 600 dense tests of $\mathrm{unpair}_2(2^i(2m+1)) = (i, m)$ for $0 \le i < 12$ and $0 \le m < 50$;
- child and input inequalities for a finite toy reflector;
- strictness of every crown edge for inputs 0 through 5;
- the exact seed–leaf–depth-zero-join–index comparison route;

- the uniform toy predecessor bound;
- a symbolic count of 43 crown edges at input 42.

The reference model is deliberately small enough to evaluate. It tests the shape of the proof and guards against off-by-one errors; it cannot certify the Lean source or the inherited F^{k}_{ω} metatheory.

13.3 Outstanding kernel checks

The environment used to produce this package did not contain `lake`. Consequently the generated validation receipt says `not_run`, rather than fabricating a pass. The included `check_in_repo.py` performs the following when run in a pinned checkout:

1. build `BusyLean.Comparison10` as the accepted predecessor;
2. time `BusyLean.Contender11` against the 15-second definition budget;
3. time `BusyLean.Comparison11` against the 60-second proof budget;
4. run non-evaluating definitional smoke tests;
5. run `#print axioms` for the new roots;
6. traverse transitive constant dependencies and reject earlier named scalar contenders.

A successful run of those commands is required before calling this an accepted BusyLean contender.

14 Integration and likely elaboration points

The proposal is additive. Copy the two modules to `BusyLean/`, append their imports to `BusyLean.lean`, merge the audit fragment, and update the checker so Contender 10 is the unbudgeted predecessor and Contender 11 is the new definition phase. The package's installer defaults to a dry run, prints unified diffs, checks the pinned Git commit, and refuses to overwrite differing candidate files.

The source closely mirrors already compiled repository patterns, but three places deserve attention during the first kernel run.

1. **Root definitional equality.** The proof of `root_program_lt_F11` uses `change` after unfolding one crown node and one forest node. If elaboration is more conservative than expected, two explicit rewriting steps should replace `change`; the theorem statement and construction need not change.
2. **Natural successor normal forms.** The crown induction uses both `k + 1` and successor induction. These are definitionally aligned in the expected Lean reduction, but a local `simpa [Nat.succ_eq_add_one]` is a harmless fallback.
3. **Timing attribution.** A full historical clean build includes all older normalization developments. The contest rule budgets the new module on top of a precompiled accepted predecessor. The supplied checker follows that three-phase discipline exactly.

No proposed repair should introduce evaluation of a contender, oracle monotonicity, new axioms, or a dependence on the named scalar.

15 Potential objections and responses

15.1 "Is the crown merely padding?"

The crown supplies a transparent diagonal margin, but the entry does not rest on it. Even before the crown, `family10_lt_stage11_index` shows that kind-polymorphic reflection along the self-generated index beats the whole predecessor family. The crown then makes the number of additional semantic levels explicit and input-dependent.

15.2 "Could Contender 10 write the same computation as one program?"

A bounded $\mathrm{F}^{\mathsf{k}}_{\omega}$ term at one reflection node may perform rich arithmetic and many oracle calls. The new construction nevertheless encloses every listed such program in a new maximum, changes the oracle to a family of recursively computed child stages, and repeats this operation along a meta-level W-tree. The exact root-program and parent-edge theorems certify that structural separation without requiring a semantic non-definability theorem for all possible encodings.

15.3 "Does the old W-language index limit the advance?"

Yes, in a precise sense: the cofinality theorem ranges over trees produced by the old W-language, because the current kind-polymorphic object language is a numeral language. This is a limitation, not a flaw in the comparison. A future object language with both kind polymorphism and positive inductive W-types could generate a stronger self-index, but its normalization and large-elimination metatheory would be a much larger formalization project.

15.4 "Where is the exact proof-theoretic ordinal?"

It is not asserted. Kind polymorphism, the precise checker, the oracle extension, forest reflection, and diagonal crown form a concrete computable hierarchy. Calibrating that hierarchy by a standard ordinal notation system would be valuable separate research. The BusyLean contest only needs the formal finite inequality, which is stronger and less ambiguous than an unproved ordinal analogy.

16 Further research program

16.1 Immediate certification

The first task is mechanical but essential: run Lean 4.34.0, repair any elaboration-only issues, record definition and proof times, and inspect the exact axiom list and dependency closure. The package separates generic lemmas from the final scalar so that repairs should not invite numerical reduction.

16.2 A kind-polymorphic W-language

The most direct qualitative successor would internalize positive inductive families or W-types into the kind-polymorphic language, with a normalizer and checker strong enough to support large elimination without inconsistency. Then the self-generated index could enumerate trees produced by the same

language that performs reflection, rather than by an older predicative language. The central research problem is to design a total calculus that is stronger than $\mathsf{F}^{\mathsf{k}}_{\omega}$ yet still admits a reducibility or logical-relations proof within the permitted Lean axiom envelope.

16.3 Typed family oracles

The current pairing exposes a family through a unary natural oracle. A richer object language could include a primitive family oracle or dependent oracle interface, eliminating coding overhead and allowing typed child data. To be a contest advance rather than syntactic sugar, one would need an exact embedding of the old interface and a program or tree not representable at bounded cost in the old syntax.

16.4 Self-generated kind indices

Contender 10 can quantify over closed kinds but does not hand a syntactic kind index back to the forest layer. One could study a two-sorted hierarchy whose nodes carry both W-tree branches and closed kind codes, with reflection over all programs and type operators below a joint budget. A diagonal index could then traverse kind order and tree order simultaneously.

16.5 Certified majorants and monotone reflection

The present proof wisely avoids oracle monotonicity. Another route is to associate each accepted program with a certified monotone majorant. Reflection could then compare entire oracle families extensionally and support more powerful transfer theorems. The challenge is obtaining useful majorants for conditionals and encoded data without collapsing the object language to an obviously weaker positive fragment.

16.6 Exact calibration

A long-term mathematical project is to identify theories whose provably total functions match the bounded maxima of the current $\mathsf{F}^{\mathsf{k}}_{\omega}$ calculus, its oracle extension, and the forest closure. Even upper and lower ordinal bounds would clarify which next language feature—large elimination, universes, inductive families, or stronger recursion—produces a genuine proof-theoretic jump.

16.7 Enumeration compression

The current syntactic enumerations grow extremely quickly and use the same input as both syntax depth and type-checking fuel. Certified binary encodings, hash-consed syntax, or direct rank enumerators might reduce checking overhead without changing the mathematical maximum. Any optimization must prove coverage and semantic agreement and must preserve the contest's finite-time build budget.

17 Conclusion

The proposed Contender 11 is

$$\boxed{C_{11} = F_{11}(F_{11}(42))}, \qquad F_{11}(n) = \mathcal{S}^{(11)}_{\mathsf{family}_{10}}(\mathsf{Crown}_{n+1}(I), n), \qquad \mathsf{family}_{10}(n) = F_{10}(F_{10}(n)).$$

It combines three mechanisms already justified separately in BusyLean: kind-polymorphic normalization, family packing, and well-founded forest reflection. The new contribution is their composition at every forest node, together with an input-diagonal crown and an exact proof architecture that needs neither oracle monotonicity nor ordinal calibration.

The mathematical comparison is concise but strong:

$$C_{10} = \mathsf{family}_{10}(42) < F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

The first inequality is backed by a leaf–join–self-index path and a strict crown; the second is the general input bound. Additional theorems expose the program and tree margins. The package is intentionally honest about its last open engineering step: the Lean files must still be compiled and audited under the pinned toolchain. All material needed for that run is included.

A Complete proposed definition module

Listing 1: `BusyLean/Contender11.lean`

```
import BusyLean.Contender10

/-!
# Contender 11: crowned kind-polymorphic forest reflection

```
family10 n    := F10 (F10 n)
F11 n         := stage11 (fun _ => family10) (crown11 (n + 1) index) n
contender_11  := F11 (F11 42)
```

`reflect11 fs n` reflects the kind-polymorphic System Fω language of
Contender 10 against the packed oracle family `fs`. `stage11` installs that
reflector at every node of a `WTree Nat`. The tree `index` is the repository's
self-generated join of every `W Nat` tree denoted by the earlier W-language.
Finally, `crown11 (n + 1) index` puts an input-diagonal unary spine above that
index, so `F11 n` crosses `n + 1` additional strict reflection levels before
reaching the self-generated forest.

The seed is Contender 10's *construction as a family*; this file never names
the scalar `contender_10`.

Do not evaluate `contender_11`.
-/

namespace BusyLean

/-- One reflection step over kind-polymorphic System Fω with a countable oracle
family. The family is encoded as one unary oracle by `packFamily`. -/
def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
```

```
    (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))

/-- Forest reflection: install `reflect11` at every node of a well-founded
countably branching tree. -/
def stage11 (seed : W.Family) : BigNumber.WTree Nat → Nat → Nat
  | .leaf => reflect11 seed
  | .node g => reflect11 (fun i => stage11 seed (g i))

/-- Put `k` unary reflection nodes above `a`.  Every node repeats the same child
at all indices; index `0` is therefore always visible. -/
def crown11 : Nat → BigNumber.WTree Nat → BigNumber.WTree Nat
  | 0, a => a
  | k + 1, a => .node fun _ => crown11 k a

/-- Contender 10's construction as a function of its input:
`family10 42 = contender_10` by definitional equality. -/
def family10 (n : Nat) : Nat := F10 (F10 n)

/-- Kind-polymorphic forest reflection along the W-language's self-generated
index, preceded by an input-diagonal reflection crown. -/
def F11 (n : Nat) : Nat :=
  stage11 (fun _ => family10) (crown11 (n + 1) index) n

/-- **Contender 11.**  Never evaluate it. -/
def contender_11 : Nat := F11 (F11 42)

end BusyLean
```

B Complete proposed comparison module

Listing 2: BusyLean/Comparison11.lean

```
import BusyLean.Contender11
import BusyLean.Comparison10

/-!
# Contender 11: comparison and structural theorems

The required result is `contender_10_lt_contender_11`.  The remaining theorems
make the margin inspectable: every visible child and every accepted
kind-polymorphic program is below its enclosing reflection node; consecutive
crown levels are strictly increasing; every W-language-defined tree is
eventually below `F11`; and the root reflects all depth-bounded programs
relative to the preceding crown level.
-/

namespace BusyLean

/-! ## One reflection step -/

theorem lt_reflect11 {fs : W.Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
      (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))) :
    m < reflect11 fs n := by
  show m < n + 1 + _
  have hM : ∀ M : Nat, M + 1 ≤ n + 1 + M := fun M => by
    rw [Nat.add_comm M 1]
```

```
    exact Nat.add_le_add_right (Nat.le_add_left 1 n) M
  exact Nat.lt_of_le_of_lt h (Nat.lt_of_lt_of_le (Nat.lt_succ_self _) (hM _))

/-- Every child visible at input `n` is strictly below the enclosing node. -/
theorem oracle_lt_reflect11 (fs : W.Family) {i n : Nat} (hi : i ≤ n) :
    fs i n < reflect11 fs n :=
  lt_reflect11 (Nat.le_trans (W.le_prefixMax (fun j => fs j n) hi)
    (BigNumber.le_maxN_left _ _))

/-- Every accepted kind-polymorphic program in the finite syntax list is below
its enclosing node. -/
theorem fvalue_lt_reflect11 (fs : W.Family) {n : Nat} {t : Fk.CTm}
    (ht : t ∈ Fk.ctmsUpTo n) :
    Fk.fvalue (packFamily fs) n t < reflect11 fs n :=
  lt_reflect11 (Nat.le_trans (W.le_maximum _ ht) (BigNumber.le_maxN_right _ _))

/-- The numerical input itself is strictly below every reflection node. -/
theorem input_lt_reflect11 (fs : W.Family) (n : Nat) : n < reflect11 fs n :=
  Nat.lt_of_lt_of_le (Nat.lt_succ_self n) (Nat.le_add_right _ _)

/-! ## Forest and crown structure -/

/-- A visible child stage is strictly below its parent stage. -/
theorem stage11_child_lt (seed : W.Family) (g : Nat → BigNumber.WTree Nat)
    {i n : Nat} (hi : i ≤ n) :
    stage11 seed (g i) n < stage11 seed (.node g) n :=
  oracle_lt_reflect11 (fun j => stage11 seed (g j)) hi

/-- Every stage strictly exceeds its numerical input. -/
theorem input_lt_stage11 (seed : W.Family) (a : BigNumber.WTree Nat) (n : Nat) :
    n < stage11 seed a n := by
  cases a with
  | leaf => exact input_lt_reflect11 seed n
  | node g => exact input_lt_reflect11 _ n

/-- The defining equation for one more crown level. -/
theorem crown11_succ (k : Nat) (a : BigNumber.WTree Nat) :
    crown11 (k + 1) a = .node (fun _ => crown11 k a) := rfl

/-- Consecutive crown levels are strictly increasing at every input. -/
theorem stage11_crown_step_lt (seed : W.Family) (a : BigNumber.WTree Nat)
    (k n : Nat) :
    stage11 seed (crown11 k a) n < stage11 seed (crown11 (k + 1) a) n := by
  rw [crown11_succ]
  exact stage11_child_lt seed (fun _ => crown11 k a) (Nat.zero_le n)

/-- Any positive crown is strictly above its base. -/
theorem stage11_lt_crown11 (seed : W.Family) (a : BigNumber.WTree Nat)
    (k n : Nat) :
    stage11 seed a n < stage11 seed (crown11 (k + 1) a) n := by
  induction k with
  | zero => simpa using stage11_crown_step_lt seed a 0 n
  | succ k ih =>
      exact Nat.lt_trans ih (stage11_crown_step_lt seed a (k + 1) n)

/-! ## The final family -/

/-- Before adding the crown, the self-generated index already dominates the
previous construction at every input. -/
theorem family10_lt_stage11_index (n : Nat) :
```

```
    family10 n < stage11 (fun _ => family10) index n := by
  have h1 : family10 n < stage11 (fun _ => family10) .leaf n :=
    oracle_lt_reflect11 (fun _ => family10) (Nat.zero_le n)
  have h2 :
      stage11 (fun _ => family10)
          (defTree (nthD (BigNumber.allTerms 0) 0 .zero)) n <
        stage11 (fun _ => family10) (treesOfDepth 0) n :=
    stage11_child_lt _ (fun j => defTree (nthD (BigNumber.allTerms 0) j .zero))
      (Nat.zero_le n)
  have h3 :
      stage11 (fun _ => family10) (treesOfDepth 0) n <
        stage11 (fun _ => family10) index n :=
    stage11_child_lt _ treesOfDepth (Nat.zero_le n)
  rw [first_tree_leaf] at h2
  exact Nat.lt_trans h1 (Nat.lt_trans h2 h3)

/-- The input-diagonal crown strictly raises the self-generated index. -/
theorem stage11_index_lt_F11 (n : Nat) :
    stage11 (fun _ => family10) index n < F11 n := by
  unfold F11
  exact stage11_lt_crown11 _ _ n n

/-- At every input, `F11` strictly exceeds Contender 10's whole family. -/
theorem family10_lt_F11 (n : Nat) : family10 n < F11 n :=
  Nat.lt_trans (family10_lt_stage11_index n) (stage11_index_lt_F11 n)

/-- Every W-language-defined tree is eventually dominated by the new family. -/
theorem defined_tree_lt_F11 {k i n : Nat} (hk : k ≤ n) (hi : i ≤ n) :
    stage11 (fun _ => family10)
        (defTree (nthD (BigNumber.allTerms k) i .zero)) n < F11 n :=
  Nat.lt_trans
    (stage11_child_lt _
      (fun j => defTree (nthD (BigNumber.allTerms k) j .zero)) hi)
    (Nat.lt_trans (stage11_child_lt _ treesOfDepth hk)
      (stage11_index_lt_F11 n))

/-- Every individual tree denoted by an enumerated W-language program is below
`F11` from an explicit input threshold onward. -/
theorem defined_tree_eventually_lt11 {k : Nat} {t : BigNumber.Term}
    (ht : t ∈ BigNumber.allTerms k) :
    ∃ N, ∀ n, N ≤ n →
      stage11 (fun _ => family10) (defTree t) n < F11 n := by
  obtain ⟨i, hi⟩ := exists_nthD_of_mem (d := BigNumber.Term.zero) ht
  refine ⟨k + i, fun n hn => ?_⟩
  rw [← hi]
  exact defined_tree_lt_F11
    (Nat.le_trans (Nat.le_add_right k i) hn)
    (Nat.le_trans (Nat.le_add_left i k) hn)

/-- At input `n`, the root of `F11 n` reflects every accepted program of depth
at most `n`, relative to the entire preceding crown level. -/
theorem root_program_lt_F11 {n : Nat} {t : Fk.CTm} (ht : t ∈ Fk.ctmsUpTo n) :
    Fk.fvalue
        (packFamily (fun _ =>
          stage11 (fun _ => family10) (crown11 n index))) n t < F11 n := by
  unfold F11
  change Fk.fvalue
      (packFamily (fun _ =>
        stage11 (fun _ => family10) (crown11 n index))) n t <
    reflect11 (fun _ =>
```

```
      stage11 (fun _ => family10) (crown11 n index)) n
  exact fvalue_lt_reflect11 _ ht

/-- **Required comparison:** Contender 10 is strictly smaller than
Contender 11.  No numerical normalization is involved. -/
theorem contender_10_lt_contender_11 : contender_10 < contender_11 :=
  Nat.lt_trans (family10_lt_F11 42)
    (input_lt_stage11 _ (crown11 (F11 42 + 1) index) (F11 42))

/-- The previous comparisons compose transitively. -/
theorem contender_9_lt_contender_11 : contender_9 < contender_11 :=
  Nat.lt_trans contender_9_lt_contender_10 contender_10_lt_contender_11

end BusyLean
```

C Reproduction checklist

1. Check out BusyLean at `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`.
2. Run `python3 integration/install.py /path/to/BusyLean` and review the printed diff.
3. Re-run with `-apply`.
4. Run `python3 scripts/check_in_repo.py /path/to/BusyLean`.
5. Require successful definition/proof phases within 15 and 60 seconds.
6. Inspect `evidence/lean_validation.external.json` and the full audit output.
7. Rebuild this article with three `pdflatex` passes from the `article/` directory.
8. Run `python3 scripts/static_audit.py` and `python3 scripts/reference_model.py`.
9. Verify `SHA256SUMS`.

D Artifact map

Path	Purpose
`candidate/BusyLean/Contender11.lean`	Computational definitions.
`candidate/BusyLean/Comparison11.lean`	Comparison and structural proofs.
`candidate/test/Audit11.lean`	Axiom and transitive dependency audit.
`candidate/test/Contender11Smoke.lean`	Non-evaluating definitional checks.
`integration/install.py`	Pinned, dry-run-first installer.
`scripts/check_in_repo.py`	Kernel, timing, and audit runner.
`scripts/static_audit.py`	Source hygiene audit.
`scripts/reference_model.py`	Independent finite structural model.
`evidence/`	Machine-readable validation receipts and build logs.
`docs/VALIDATION_BOUNDARY.md`	Explicit trust boundary.
`article/contender11.tex`	This source.

Path	Purpose
`article/contender11.pdf`	Compiled report.

References

[1] BusyLean contributors. *BusyLean: name the biggest number, in Lean*. Pinned revision `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`, October 2026. https://github.com/VladimirReshetnikov/BusyLean/tree/4c4b03d0e2003c479e3bee90a21c8bea13f66c40.

[2] Cody Roux and contributors. *name-the-biggest-number*. https://github.com/codyroux/name-the-biggest-number.

[3] Scott Aaronson. *Who Can Name the Bigger Number?* https://www.scottaaronson.com/writings/bignumbers.html.

[4] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur*. Thèse d'État, Université Paris VII, 1972.

[5] John C. Reynolds. Towards a theory of type structure. In *Programming Symposium*, LNCS 19, pages 408–425, 1974.

[6] Henk Barendregt. Lambda calculi with types. In Samson Abramsky, Dov Gabbay, and Thomas Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.

[7] Antonius J. C. Hurkens. A simplification of Girard's paradox. In *Typed Lambda Calculi and Applications*, LNCS 902, pages 266–278, 1995.

[8] The Lean development team. *The Lean Language Reference: Recursive Definitions*. https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/. Consulted October 2026; the package itself pins Lean 4.34.0.